Academia\BackEnd\Academia Staff\Academia_Staff.c

```c
/*
Harshul Gupta, Lawrence Mao, Suengho Kim, Dmitrii Buianov
CS50 C Project
Academia
November 1, 2026
*/

#include "Academia_Staff.h"
#include "../Database.h"

#include <stdio.h>
#include <string.h>

#ifndef _WIN32
#include <sqlite3.h>
#endif

<<<<<<< HEAD
/* write_list runs one query and writes the rows as a JS array */
static void write_list(FILE *out, sqlite3 *db, const char *name, const char *sql, int staff_id) {
    sqlite3_stmt *row = NULL;
=======
/* escapes quotes so a name like Ann "Annie" Lee cannot break the file */
static void write_js_text(FILE *out, const unsigned char *text) {
    int i;
>>>>>>> 343e1edff3a9b596c6861bf939d4762787ad76f4

    /* start the array */
    fprintf(out, "  %s: [\n", name);

    /* if the query will not prepare, close the array so the page still parses */
    if (sqlite3_prepare_v2(db, sql, -1, &row, NULL) != SQLITE_OK) {
        fputs("  ],\n", out);
        return;
    }
    /* bind the id so the query can never be injected */
    sqlite3_bind_int(row, 1, staff_id);

    /* one line of JavaScript per row */
    while (sqlite3_step(row) == SQLITE_ROW) {
        int i;
        fputs("    {", out);
        for (i = 0; i < sqlite3_column_count(row); i++) {
            const char *text = (const char *)sqlite3_column_text(row, i);

            fprintf(out, "%s%s: ", i ? ", " : "", sqlite3_column_name(row, i));

```

```
            /* numbers go in as numbers, everything else as a quoted string */
            if (sqlite3_column_type(row, i) == SQLITE_FLOAT) {
                fprintf(out, "%.1f", sqlite3_column_double(row, i));
            } else if (sqlite3_column_type(row, i) == SQLITE_INTEGER) {
                fprintf(out, "%d", sqlite3_column_int(row, i));
            } else {
                fprintf(out, "\"%s\"", text == NULL ? "" : text);
            }
        }
        fputs(" },\n", out);
    }

    sqlite3_finalize(row);
    fputs("  ],\n", out);
}

/* update_staff_frontend_state builds staff_states.js for the faculty portal */
int update_staff_frontend_state(int staff_user_id) {
    sqlite3 *db = open_database();
    sqlite3_stmt *row = NULL;
    /* found stays 0 until the staff row is seen */
    FILE *out = NULL;
    int found = 0;

    if (!db) {
        printf("[Staff] Cannot open the database.\n");
        return 0;
    }

    // Join the general user account with its staff-specific information.
    const char *sql =
        "SELECT user.id, user.username, user.first_name, user.last_name, staff.phone "
        "FROM user "
        "INNER JOIN staff ON user.id = staff.id "
        "WHERE user.id = ? AND user.role = 'staff';";

    /* staff profile */
    /* profile block, written column by column */
    fputs("  staff: {\n", out);
    if (sqlite3_prepare_v2(db,
            "SELECT user.username AS username, user.first_name AS firstName, "
            "user.last_name AS lastName, staff.phone AS phone FROM user "
            "INNER JOIN staff ON user.id = staff.id WHERE user.id = ?;",
            -1, &row, NULL) == SQLITE_OK) {
        sqlite3_bind_int(row, 1, staff_user_id);
        if (sqlite3_step(row) == SQLITE_ROW) {
            int i, n = sqlite3_column_count(row);
            found = 1;
            fprintf(out, "    userId: %d,\n", staff_user_id);
            for (i = 0; i < n; i++) {
```

```
                const char *text = (const char *)sqlite3_column_text(row, i);
                fprintf(out, "    %s: \"%s\"%s\n", sqlite3_column_name(row, i),
                        text == NULL ? "" : text, i == n - 1 ? "" : ",");
            }
        }
        sqlite3_finalize(row);
    }

    // Use the signed-in staff member's ID in the query.
    sqlite3_bind_int(statement, 1, staff_id);

    if (sqlite3_step(statement) == SQLITE_ROW) {
        fputs("    userId: ", out);
        fprintf(out, "%d", sqlite3_column_int(statement, 0));
        fputs(",\n", out);

        fputs("    username: ", out);
        write_js_quoted(out, sqlite3_column_text(statement, 1));
        fputs(",\n", out);

        fputs("    firstName: ", out);
        write_js_quoted(out, sqlite3_column_text(statement, 2));
        fputs(",\n", out);

        fputs("    lastName: ", out);
        write_js_quoted(out, sqlite3_column_text(statement, 3));
        fputs(",\n", out);

        fputs("    phone: ", out);
        write_js_quoted(out, sqlite3_column_text(statement, 4));
        fputs("\n", out);
    }

    sqlite3_finalize(statement);
    fputs("  },\n", out);

    /* the teacher's courses, with active student counts */
    /* only the courses this teacher runs */
    write_list(out, db, "courses",
        "SELECT course.id AS courseId, course.course_name AS course, "
        "(SELECT COUNT(*) FROM enrollment WHERE enrollment.course_id = course.id "
        "AND enrollment.status = 1) AS students FROM course "
        "WHERE course.staff_id = ? ORDER BY course.id;", staff_user_id);

<<<<<<< HEAD
    /* roster with the current grade */
    /* roster of those courses, with the current grade */
    write_list(out, db, "students",
        "SELECT user.id AS studentId, user.first_name || ' ' || user.last_name AS name, "
        "user.username AS username, course.course_name AS course, "
```

```
        "(SELECT score FROM grade WHERE grade.student_id = user.id "
        "AND grade.course_id = course.id) AS score FROM enrollment "
        "INNER JOIN student ON enrollment.student_id = student.id "
        "INNER JOIN user ON student.id = user.id "
        "INNER JOIN course ON enrollment.course_id = course.id "
        "WHERE course.staff_id = ? ORDER BY user.last_name;", staff_user_id);

    /* attendance for the teacher's courses */
=======
    const char *sql =
        "SELECT id, course_name FROM course WHERE staff_id = ?;";

    fputs("  courses: [\n", out);

    if (sqlite3_prepare_v2(db, sql, -1, &statement, NULL) != SQLITE_OK) {
        fputs("  ],\n", out);
        return;
    }

    sqlite3_bind_int(statement, 1, staff_id);

    // Write every course assigned to this staff member.
    while (sqlite3_step(statement) == SQLITE_ROW) {
        fputs("    { courseId: ", out);
        fprintf(out, "%d", sqlite3_column_int(statement, 0));

        fputs(", course: ", out);
        write_js_quoted(out, sqlite3_column_text(statement, 1));

        fputs(" },\n", out);
    }

    sqlite3_finalize(statement);
    fputs("  ],\n", out);
}

/* write_enrollments_section Writes one row per student on each of this teacher's courses */
static void write_enrollments_section(
    FILE *out,
    sqlite3 *db,
    int staff_id
) {
    sqlite3_stmt *statement = NULL;

    /* The two pipe symbols join two text columns into one full name, so the C code receives
*/
    const char *sql =
        "SELECT DISTINCT user.id, "
        "       user.first_name || ' ' || user.last_name, "
        "       course.id, course.course_name "
```

```
        "FROM enrollment "
        "INNER JOIN user ON enrollment.student_id = user.id "
        "INNER JOIN course ON enrollment.course_id = course.id "
        "WHERE course.staff_id = ? ORDER BY user.last_name;", staff_user_id);

<<<<<<< HEAD
    fputs("  enrollments: [\n", out);

    if (sqlite3_prepare_v2(db, sql, -1, &statement, NULL) != SQLITE_OK) {
        fputs("  ],\n", out);
        return;
    }

    sqlite3_bind_int(statement, 1, staff_id);

    // Each row connects one student to one of the teacher's courses.
    while (sqlite3_step(statement) == SQLITE_ROW) {
        fputs("    { studentId: ", out);
        fprintf(out, "%d", sqlite3_column_int(statement, 0));

        fputs(", student: ", out);
        write_js_quoted(out, sqlite3_column_text(statement, 1));

        fputs(", courseId: ", out);
        fprintf(out, "%d", sqlite3_column_int(statement, 2));

        fputs(", course: ", out);
        write_js_quoted(out, sqlite3_column_text(statement, 3));

        fputs(" },\n", out);
    }

    sqlite3_finalize(statement);
    fputs("  ],\n", out);
}

/* write_grades_section Writes every grade recorded in this teacher's courses, so the gr */
static void write_grades_section(FILE *out, sqlite3 *db, int staff_id) {
    sqlite3_stmt *statement = NULL;

    // Filter through course.staff_id so only this teacher's grades are shown.
    const char *sql =
        "SELECT grade.id, "
        "       user.first_name || ' ' || user.last_name, "
        "       course.course_name, grade.score "
        "FROM grade "
        "INNER JOIN user ON grade.student_id = user.id "
        "INNER JOIN course ON grade.course_id = course.id "
        "WHERE course.staff_id = ?;";

```

```
    fputs("  grades: [\n", out);

    if (sqlite3_prepare_v2(db, sql, -1, &statement, NULL) != SQLITE_OK) {
        fputs("  ],\n", out);
        return;
    }

    sqlite3_bind_int(statement, 1, staff_id);

    while (sqlite3_step(statement) == SQLITE_ROW) {
        fputs("    { gradeId: ", out);
        fprintf(out, "%d", sqlite3_column_int(statement, 0));

        fputs(", student: ", out);
        write_js_quoted(out, sqlite3_column_text(statement, 1));

        fputs(", course: ", out);
        write_js_quoted(out, sqlite3_column_text(statement, 2));

        fputs(", score: ", out);
        fprintf(out, "%.1f },\n", sqlite3_column_double(statement, 3));
    }

    sqlite3_finalize(statement);
    fputs("  ],\n", out);
}

/* write_attendance_section Writes every attendance record for this teacher's courses */
static void write_attendance_section(
    FILE *out,
    sqlite3 *db,
    int staff_id
) {
    sqlite3_stmt *statement = NULL;

    // The course table identifies which attendance records belong to staff.
    const char *sql =
        "SELECT attendance.id, "
        "       user.first_name || ' ' || user.last_name, "
        "       course.course_name, attendance.present "
        "FROM attendance "
        "INNER JOIN user ON attendance.student_id = user.id "
=======
>>>>>>> 343e1edff3a9b596c6861bf939d4762787ad76f4
    /* attendance for the same courses */
    write_list(out, db, "attendance",
        "SELECT course.course_name AS course, "
        "user.first_name || ' ' || user.last_name AS student, "
        "attendance.present AS present FROM attendance "
>>>>>>> 4589ef827824c44d3008ef83abaa599c2ef01fe9
```

```
        "INNER JOIN course ON attendance.course_id = course.id "
        "INNER JOIN student ON attendance.student_id = student.id "
        "INNER JOIN user ON student.id = user.id "
        "WHERE course.staff_id = ? ORDER BY course.id;", staff_user_id);

    /* announcements for the teacher's courses */
    write_list(out, db, "announcements",
        "SELECT course.course_name AS course, announcement.title AS title, "
        "announcement.message AS message, announcement.announcement_date AS date "
        "FROM announcement INNER JOIN course ON announcement.course_id = course.id "
        "WHERE course.staff_id = ? ORDER BY announcement.id DESC;", staff_user_id);

    fprintf(out, "  success: %s,\n  staffId: %d\n};\n",
            found ? "true" : "false", staff_user_id);

    sqlite3_bind_int(statement, 1, staff_id);

    while (sqlite3_step(statement) == SQLITE_ROW) {
        fputs("    { attendanceId: ", out);
        fprintf(out, "%d", sqlite3_column_int(statement, 0));

        fputs(", student: ", out);
        write_js_quoted(out, sqlite3_column_text(statement, 1));

        fputs(", course: ", out);
        write_js_quoted(out, sqlite3_column_text(statement, 2));

        /* The JavaScript expects a number here: 1 for present, 0 for absent */
        fputs(", present: ", out);
        fprintf(out, "%d", sqlite3_column_int(statement, 3));

        fputs(" },\n", out);
    }

    sqlite3_finalize(statement);
    fputs("  ],\n", out);
}

/* write_announcements_section Writes the announcements, newest first */
static void write_announcements_section(FILE *out, sqlite3 *db) {
    sqlite3_stmt *statement = NULL;

    // LEFT JOIN keeps general announcements that have no linked course.
    const char *sql =
        "SELECT announcement.id, course.course_name, announcement.title, "
        "       announcement.message, announcement.announcement_date "
        "FROM announcement "
        "LEFT JOIN course ON announcement.course_id = course.id "
        "ORDER BY announcement.id DESC;";

```

```
    fputs("  announcements: [\n", out);

    if (sqlite3_prepare_v2(db, sql, -1, &statement, NULL) != SQLITE_OK) {
        fputs("  ],\n", out);
        return;
    }

    while (sqlite3_step(statement) == SQLITE_ROW) {
        const unsigned char *course =
            sqlite3_column_text(statement, 1);

        fputs("    { announcementId: ", out);
        fprintf(out, "%d", sqlite3_column_int(statement, 0));

        fputs(", course: ", out);

        // Use General when the announcement is not tied to a course.
        if (course == NULL) {
            write_js_quoted(
                out,
                (const unsigned char *)"General"
            );
        } else {
            write_js_quoted(out, course);
        }

        fputs(", title: ", out);
        write_js_quoted(out, sqlite3_column_text(statement, 2));

        fputs(", message: ", out);
        write_js_quoted(out, sqlite3_column_text(statement, 3));

        fputs(", date: ", out);
        write_js_quoted(out, sqlite3_column_text(statement, 4));

        fputs(" },\n", out);
    }

    sqlite3_finalize(statement);
    fputs("  ],\n", out);
}

/* update_staff_frontend_state is the function the server calls */
void update_staff_frontend_state(int staff_user_id) {
    sqlite3 *db = NULL;
    FILE *out = NULL;

    /* Step 1: open the database */
    if (sqlite3_open("academia.db", &db) != SQLITE_OK) {
        printf("[Staff Module] Cannot open academia.db\n");
```

```
397
398         if (db != NULL) {
399             sqlite3_close(db);
400         }
401
402         return;
403     }
404
405     /* Step 2: open the file the portal reads */
406     out = fopen("FrontEnd/JS/staff_states.js", "w");
407
408     // Try the backend path if the first output path is unavailable.
409     if (out == NULL) {
410         out = fopen(
411             "BackEnd/Academia Staff/staff_states.js",
412             "w"
413         );
414     }
415
416     if (out == NULL) {
417         printf(
418             "[Staff Module] Cannot open staff_states.js for writing.\n"
419         );
420
421         sqlite3_close(db);
422         return;
423     }
424
425     fputs(
426         "// Auto-generated by the Academia C backend. "
427         "Do not edit by hand.\n",
428         out
429     );
430
431     fputs("const staffState = {\n", out);
432
433     // Write each section of the Staff Portal state.
434     write_staff_section(out, db, staff_user_id);
435     write_courses_section(out, db, staff_user_id);
436     write_enrollments_section(out, db, staff_user_id);
437     write_grades_section(out, db, staff_user_id);
438     write_attendance_section(out, db, staff_user_id);
439     write_announcements_section(out, db);
440
441     /* the portal checks this flag */
442     fputs("  success: true\n", out);
443     fputs("};\n", out);
444
445     /* Step 3: close both */
446     fclose(out);
```

```
447     sqlite3_close(db);
448 
449     printf(
450         "[Staff Module] staff_states.js written for staff %d\n",
451         staff_user_id
452     );
453 }
454 
455 /* WRITE HALF (new) */
456 
457 /* returns 1 only if this teacher owns the course */
458 static int staff_owns_course(sqlite3 *db, int staff_id, int course_id) {
459     /* no write is allowed unless this is true */
460     sqlite3_stmt *s = NULL;
461     int ok = 0;
462     if (sqlite3_prepare_v2(db, "SELECT 1 FROM course WHERE id = ? AND staff_id = ?;",
463                            -1, &s, NULL) != SQLITE_OK) return 0;
464     sqlite3_bind_int(s, 1, course_id);
465     sqlite3_bind_int(s, 2, staff_id);
466     ok = (sqlite3_step(s) == SQLITE_ROW);
467     sqlite3_finalize(s);
468     return ok;
469 }
470 
471 /* returns 1 only if the student is actively enrolled in the course */
472 static int student_enrolled(sqlite3 *db, int student_id, int course_id) {
473     /* guards the grade and attendance saves below */
474     sqlite3_stmt *s = NULL;
475     int ok = 0;
476     if (sqlite3_prepare_v2(db,
477             "SELECT 1 FROM enrollment WHERE student_id = ? AND course_id = ? AND status =
    1;",
478             -1, &s, NULL) != SQLITE_OK) return 0;
479     sqlite3_bind_int(s, 1, student_id);
480     sqlite3_bind_int(s, 2, course_id);
481     ok = (sqlite3_step(s) == SQLITE_ROW);
482     sqlite3_finalize(s);
483     return ok;
484 }
485 
486 /* saves one grade and refreshes the student's GPA */
487 int staff_save_grade(int staff_id, int student_id, int course_id, double score) {
488     /* update the row if there is one */
489     sqlite3 *db = open_database();
490     sqlite3_stmt *s = NULL;
491     int ok = 0;
492 
493     if (!db) return 0;
494     if (score < 0 || score > 100 ||
495         !staff_owns_course(db, staff_id, course_id) ||
```

```
        !student_enrolled(db, student_id, course_id)) {
        close_database(db);
        return 0;
    }

    if (sqlite3_prepare_v2(db, "UPDATE grade SET score = ? WHERE student_id = ? AND course_id = ?;",
                           -1, &s, NULL) == SQLITE_OK) {
        sqlite3_bind_double(s, 1, score);
        sqlite3_bind_int(s, 2, student_id);
        sqlite3_bind_int(s, 3, course_id);
        sqlite3_step(s);
        sqlite3_finalize(s);
    }
    if (sqlite3_changes(db) == 0) {
        if (sqlite3_prepare_v2(db, "INSERT INTO grade (student_id, course_id, score) VALUES (?, ?, ?);",
                               -1, &s, NULL) == SQLITE_OK) {
            sqlite3_bind_int(s, 1, student_id);
            sqlite3_bind_int(s, 2, course_id);
            sqlite3_bind_double(s, 3, score);
            ok = (sqlite3_step(s) == SQLITE_DONE);
            sqlite3_finalize(s);
        }
    } else {
        ok = 1;
    }

    if (ok && sqlite3_prepare_v2(db,
            "UPDATE student SET gpa = (SELECT AVG(score) / 25.0 FROM grade WHERE student_id = ?) "
            "WHERE id = ?;", -1, &s, NULL) == SQLITE_OK) {
        sqlite3_bind_int(s, 1, student_id);
        sqlite3_bind_int(s, 2, student_id);
        sqlite3_step(s);
        sqlite3_finalize(s);
    }

    close_database(db);
    return ok;
}

/* saves present or absent for one student in one course */
int staff_save_attendance(int staff_id, int student_id, int course_id, int present) {
    /* attendance has no date column, so each save replaces the last */
    sqlite3 *db = open_database();
    sqlite3_stmt *s = NULL;
    int ok = 0;

    if (!db) return 0;
    if (!staff_owns_course(db, staff_id, course_id) ||
```

```
        !student_enrolled(db, student_id, course_id)) {
        close_database(db);
        return 0;
    }

    if (sqlite3_prepare_v2(db, "UPDATE attendance SET present = ? WHERE student_id = ? AND course_id = ?;",
                           -1, &s, NULL) == SQLITE_OK) {
        sqlite3_bind_int(s, 1, present ? 1 : 0);
        sqlite3_bind_int(s, 2, student_id);
        sqlite3_bind_int(s, 3, course_id);
        sqlite3_step(s);
        sqlite3_finalize(s);
    }
    if (sqlite3_changes(db) == 0) {
        if (sqlite3_prepare_v2(db, "INSERT INTO attendance (course_id, student_id, present) VALUES (?, ?, ?);",
                               -1, &s, NULL) == SQLITE_OK) {
            sqlite3_bind_int(s, 1, course_id);
            sqlite3_bind_int(s, 2, student_id);
            sqlite3_bind_int(s, 3, present ? 1 : 0);
            ok = (sqlite3_step(s) == SQLITE_DONE);
            sqlite3_finalize(s);
        }
    } else {
        ok = 1;
    }

    close_database(db);
    return ok;
}
```